

Programowanie Komputerów 4

Temat: Oczko – Black Jack

Autor: Kacper Nitkiewicz

Rok: 2020/2021

Rodzaj studiów: [SSI]

Semestr: 4

Sekcja: 51

Termin oddania: 26-05-2021

1 Treść zadania

Program będzie odpalany przy użyciu konsoli. Przy otwarciu będziemy musieli podać ilość graczy jacy będą uczestniczyć w grze. Następnie pojawi się stół na którym każdy z graczy będzie musiał potwierdzić, w zależności o stanu swojego konta, ilość pieniędzy jaką chce przeznaczyć na daną rundę. Każdy grający musi taką czynność wykonać lub odejść od stołu. Następnie krupier (komputer), rozdaje każdemu dwie karty i gracz decyduje czy chce grać dalej (dołożyć kartę), podwoić stawkę (otrzymać ostatnią kartę i podwoić wcześniej postawione pieniądze) lub spasować. W zależności od decyzji gracz krupier dodaje lub przestaje pytać gracza o następne karty. Jeżeli gracz przy 3 karcie wyrzuci >21 punktów to dostaje informację o wyrzuceniu fury i nie gra do końca rundy. (punkty na podstawie podstawowej gry w oczko (<https://en.wikipedia.org/wiki/Blackjack> -> rules)). Gracz może spasować po 2 kartach, jeśli ma taką ochotę. Jeżeli choć jedna z osób spasuje przed wyrzuceniem fury to krupier wykonuje swoje rzuty. Jeżeli gracz i krupier posiadają identyczną liczbę oczek to pieniądze wyłożone przez gracza wracają do niego. Jeżeli gracz wyrzuci 21 w pierwszych 2 kartach o otrzymuje połowę swojego zakładu i nie rywalizuje z krupierem w tej rundzie (np. dał 6 to otrzyma 9 na koniec). Jeżeli gracz wyrzuci 21 w kolejnych rozdaniach to otrzymuje podwójną stawkę (chyba, że krupier też trafi 21). Jeżeli ilość punktów gracza > ilość punktów krupiera to gracz otrzymuje podwójną ilość swojego zakładu. W przeciwnym wypadku krupier zabiera pieniądze postawione przez gracza. Gracz może odejść od stołu przed postawieniem pieniędzy. Hasz obok krupiera oznacza zasłoniętą kartę (może %).

2 Analiza rozwiązania

Możliwość rozpoczęcia gry od nowa lub kontynuacji poprzedniej rozgrywki. Przy wyborze nowej rozgrywki podajemy nowych graczy oraz ich pieniądze jakie posiadają. Po tym przechodzimy do drugiego ekranu w którym każdy z zawodników może postawić pieniądze. Po postawieniu pieniędzy przez każdego zawodnika zostają rozdane karty. Zawodnik może spasować, dobrać kartę albo podwoić ilość postawionych pieniędzy w zamian za jedną kartę.

3 Specyfikacja zewnętrzna

Program należy uruchomić podając nazwę pliku Proj.exe. Następnie otwarty zostanie okno graficzne w którym wszystkie rzeczy będą pokazywane graczowi. W oknie Zero gracz może wybrać czy chce zacząć nową grę czy przywrócić poprzednią rozgrywkę. W pierwszym oknie jeżeli wybrano nową rozgrywkę to można podać nazwy gracza oraz ilość pieniędzy. W trzecim ekranie dostajemy dwie karty i można zdecydować o losie gracza przez naciśnięcie guzików podanych na ekranie.

4 Specyfikacja wewnętrzna

Presentation.h:

- Struct Presentation; - główna classa wykonująca program.
- Void main(); - metoda wykonująca program;

Entity.h:

- class Entity- klasa osoby posiada trzy pola- ilość pokazanych kart osobie, imie i punkty (z sumy kart) oraz następujące metody:
 - Entity(): points(0),cards_shown(2) – konstruktor tworzący
 - Entity(string _name) :name(_name),points(0),cards_shown(2); - konstruktor tworzący z imieniem.
 - string get_name() – dostarcza imie entity
 - int get_points() – dostarcza punkty entity
 - void add_points(int p) – dodaje punkty do points
 - void reset_points_cards() – resetuje punkty aby były czyste dla następnego okna
 - void increm_cards_shown() – dodaje jedną kartę do cards_shown
 - int get_cards_shown() – dostarcza ilość pokazanych kart
- Reszta to metody wirtualne.

History.h:

- Class History- klasa historii rozgrywki, zamieszczona w Entity posiada historie poprzedniej rozgrywki. Klasa ma dwa wektory z których korzysta: names – imiona graczy, prize_pool – ilość pieniędzy jaką wygrali/przegrali.
- void add(string name, int prize); - dodaje do wektorów nowe dane.
- Void clear(); - czyści wektory w klasie.
- Bool empty(); - sprawdza czy historia jest pusta – czy wektory są puste.
- String result; - dostarcza napis, który posiada wszystkie informacje o historii rozgrywki na podstawie posiadanych wektorów.

Dealer.h:

- Class Dealer- klasa dziedzicząca z Entity, gracz komputerowy.
- Konstruktory dealera tworzą Entity o nazwie „Dealer” oraz dodają income lub ustawiają go na zero.
- int calculate_income(vector<Entity*> ent); - oblicza income jaki dealer otrzyma.
- Get oraz set j.w. – tylko zapisuje do income oraz czyta income.

Player.h:

- Class Player- klasa dziedzicząca z Entity, gracz rzeczywisty.
- Konstruktory playera tworzą Entity o nazwie podanej przez gracza oraz dodają pieniądze dla niego.
- Get oraz set j.w. – tylko zapisuje do bet/money oraz czyta bet/money.

Card.h:

- Class Card- tworzy kartę posiadającą teksturę, rozmiar pojedynczej tekstury oraz miejsce tekstury w którym jest czytana. Posiada również nazwę oraz wektor nazw wszystkich kart.

- Konstruktor tworzy kartę o ustalonym wcześniej rozmiarze.
- `vector<string> vec_name_cards();` - funkcja tworzy wektor wszystkich kart możliwych do zaistnienia.
- `bool is_card();` - sprawdza czy jest to karta na podstawie imienia.
- `void random();` - tworzy losową kartę z pośród 52 możliwości.
- `void random(string turned);` - tworzy losową kartę która jest obrócona figurą do dołu czyli nie widoczna dla gracza.
- `sf::RectangleShape get_Shape();` - zwraca `sf::RectangleShape` dla danej karty.

Windows.h:

- Class `Windows`- bazowa klasa okna, posiadająca wirtualny `screen()` z której to metody korzystają funkcje dziedziczące.
- `vector<Entity*> createEntities(vector<string> names, vector<int> values);` - tworzy wektor `Entity` z posiadanych imion oraz wartości pieniędzy.

Zero.h:

- Class `Zero`- klasa dziedzicząca z `Windows`. Zerowy (początkowy) ekran rozgrywki.
- `void Screen(sf::RenderWindow& window, vector<Entity*>& vec_ent);` - metoda sprawdza czy gracz chce zacząć nową rozgrywkę czy kontynuować poprzednią.
- `void load_from_file(vector<string>& vec_str, vector<int>& vec_int);` - ładuje z pliku do wektorów odpowiednie wartości jeżeli wybrano kontynuację.
- `void methodY(vector<Entity*>& vec_ent);` - metoda wykonuje szereg zadań jeżeli wybrano kontynuację.

First.h:

- Class `First`- klasa dziedzicząca z `Windows`. Pierwszy ekran rozgrywki.
- `void Screen(sf::RenderWindow& window, vector<Entity*>& vec_ent) override;` - metoda pobiera od gracza nazwy oraz ilości pieniędzy z wyświetlaniem na ekranie.
- `vector<int> vec_values(const string&);` - tworzy wektor wartości z otrzymanego ciągu znaków.
- `vector<string> vec_names(const string& names);` - tworzy wektor imion z otrzymanego ciągu znaków.

Second.h:

- Class `Second`- klasa dziedzicząca z `Windows`. Drugi ekran rozgrywki.
- `void Screen(sf::RenderWindow& window, vector<Entity*>& vec_ent) override;` - pozwala graczom na wybranie betu.

Third.h:

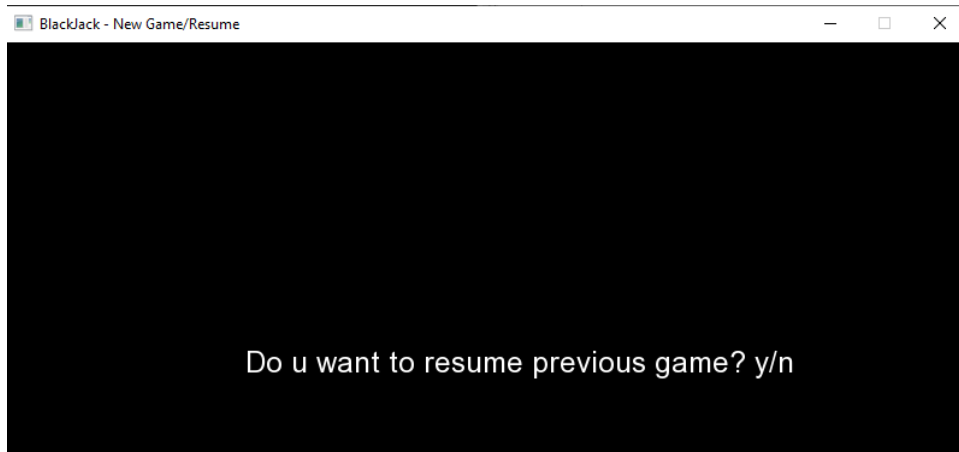
- Class `Third`- klasa dziedzicząca z `Windows`. Trzeci ekran rozgrywki.
- `void Screen(sf::RenderWindow& window, vector<Entity*>& vec_ent) override;` - funkcja pozwala graczem na wybranie decyzji na podstawie otrzymanych kart. Dealer następnie podejmuje decyzje i gra wraca do drugiego ekranu.
- `int hit_or_stand(int casey);` - decyzja dealera aby dobrać kartę albo odpuścić dobieranie.
- `int calculate_card(string x);` - oblicza ile warta jest karta. Od 2 do 11.
- `void save_to_file(vector<Entity*>& vec_ent);` - zapisuje do pliku ilość pieniędzy jakie posiada gracz.
- `void hitButton(vector<Entity*>& vec_ent, int current_player, vector<Card>& cards);` - dobranie karty przez gracza.

- void doubledownButton(int& current_player, vector<Entity*>& vec_ent); - podwojenie betu przez gracza I dobranie jednej karty.
- void calculate_start_cards(vector<Entity*>& vec_ent,vector<Card> cards); - oblicza wszystkie początkowe karty graczy.

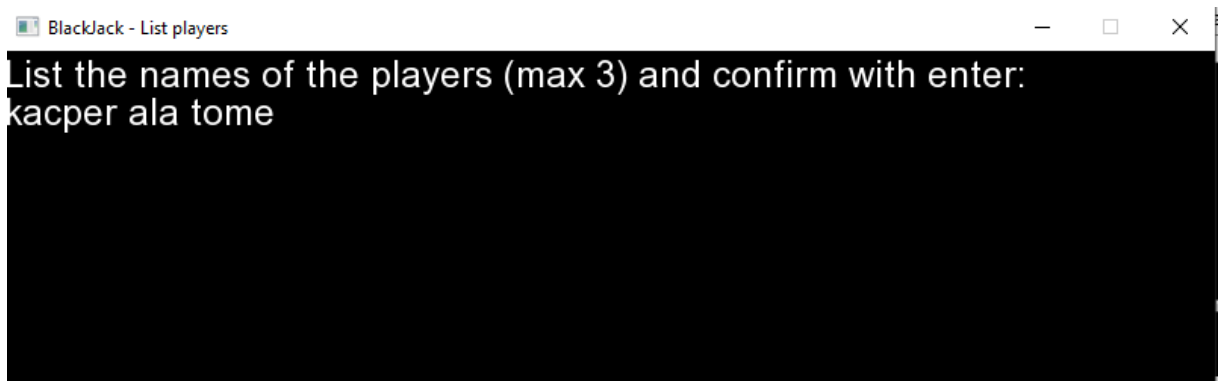
5 Testowanie

Przykład 1.

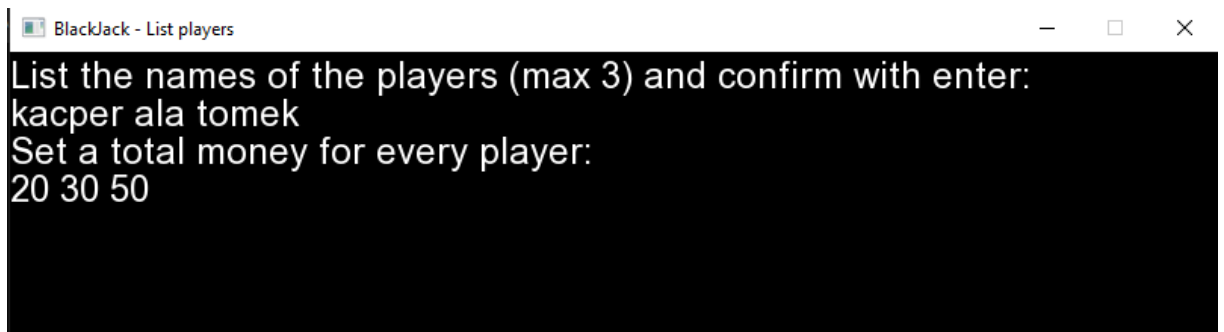
Wybieramy czy chcemy zacząć rozgrywkę od nowa przeciskiem N.



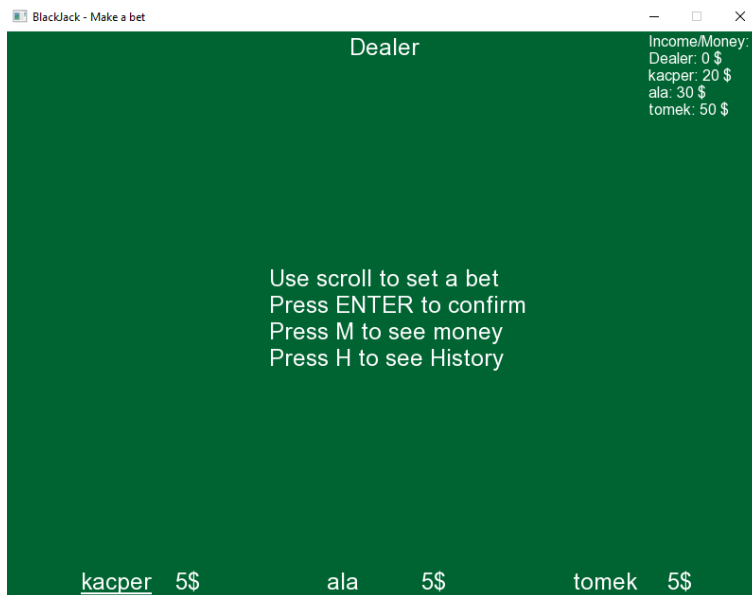
Podajemy imiona graczy:



Podajemy ilość pieniędzy dla każdego:

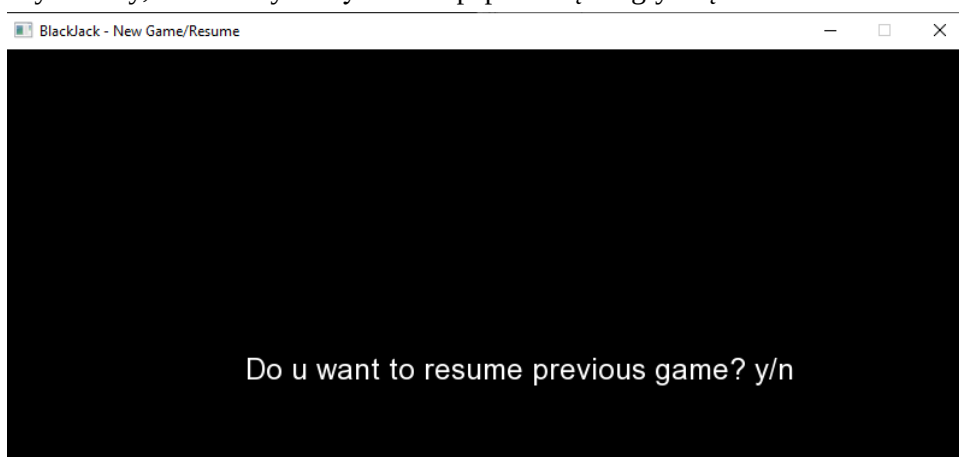


Otrzymujemy poprawny rezultat.

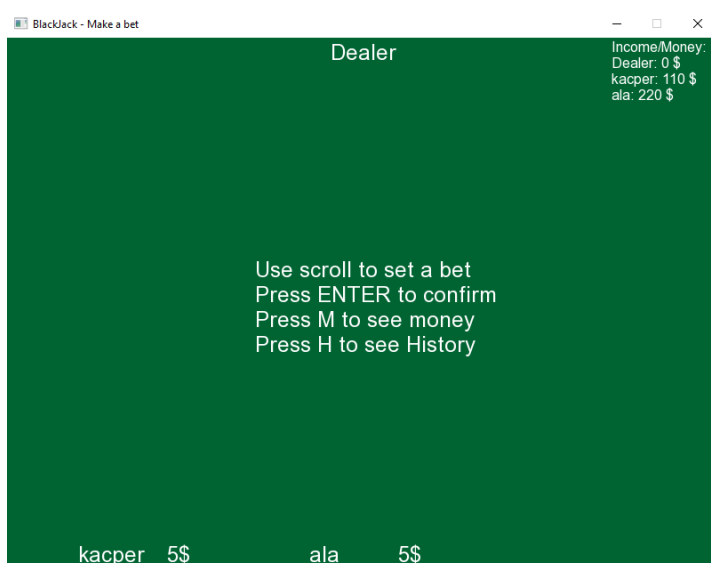


Przykład 2.

Wybieramy, że chcemy kontynuować poprzednią rozgrywkę klawiszem Y.

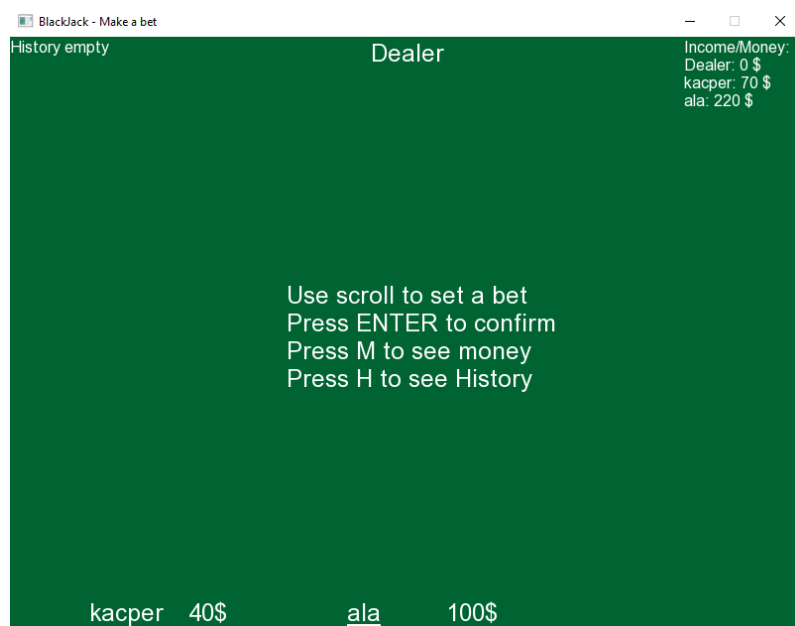


Z pliku wczytywane są parametry poprzedniej rozgrywki.

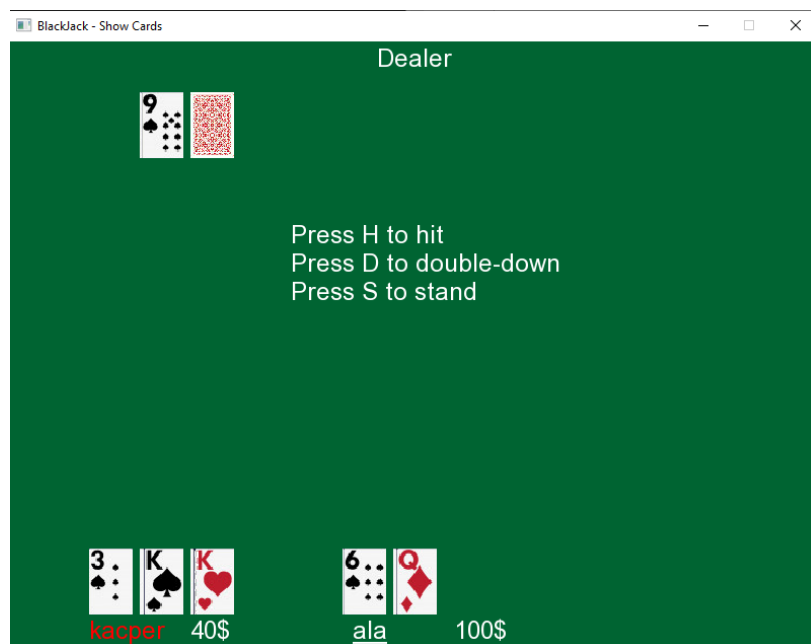


Przykład 2.

Możemy ustawić bety każdego zawodnika scrolllem. Jeżeli bet będzie większy od pieniędzy nie pozwoli nam go potwierdzić. Można sprawdzić historie literą H.



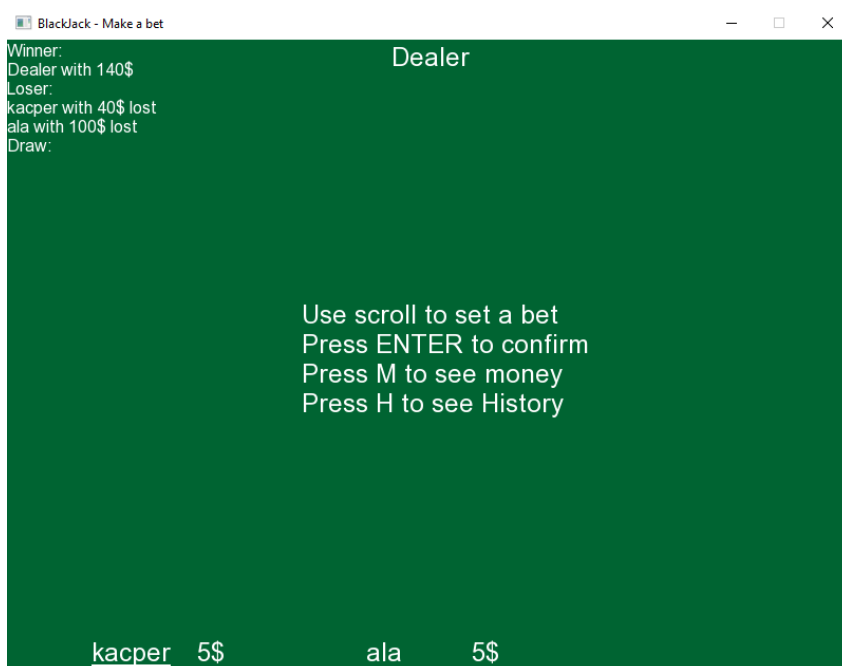
Po zatwierdzeniu obu betów możemy rozpocząć rozgrywkę i pokazać karty. Gracz kacper posiada 13 punktów po rozdaniu 2 kart. Decyduje się dobrać - Hit.



Niestety przegrywa (bust) ponieważ posiada 23 punkty. Kolej na Ale. Ona decyduje się zostać przy 16 punktach – Stand.



Niestety dealer posiada 19 punktów więc wygrywa z Kacper i z Alą zabierając ich pieniądze. Można sprawdzić historię poprzedniej rozgrywki po naciśnięciu Enter i H.



Gracze mogą zacząć następną rozgrywkę.